

Programmation Multiplateforme

React Native

Table des matières

Créez votre première application React Native.....	2
Installation.....	2
Créer un nouveau projet.....	2
Tester le nouveau projet.....	2
Structure d'un projet React Native.....	3
Les composants.....	3
Mise en forme d'un composant.....	4
Mise en forme avec Flex Box.....	5
Propriétés d'un composant.....	9
Propriété onPress.....	10
États d'un composant.....	12
Communication enfant-parent (<i>lifting state up</i>).....	13
Exemple complet.....	15
Navigation entre les vues.....	21
Installer React Navigation.....	21
Navigation par pile (<i>StackNavigator</i>).....	22
Passage de paramètres à une route.....	24
Cycle de vie des composants.....	25
Navigation par menu (<i>DrawerNavigator</i>).....	27
Navigation par onglet (<i>TabNavigator</i>).....	29
Persistance des données.....	30
Géolocalisation.....	32
Base de données locale (<i>mongoose like</i>).....	34
Base de données distante.....	38
MQTT.....	38
Publier sur Google Play.....	40
Références.....	40

Document écrit par Stéphane Gill

Mars 2022



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

React Native est un cadre d'applications open source créé par Facebook. Il est utilisé pour développer des applications pour Android, iOS et UWP en permettant aux développeurs d'utiliser *React* avec les fonctionnalités natives de ces plateformes.

Créez votre première application React Native

Installation

- Installer *Node.js*
- Installer JDK
- Installer *Android Studio* et configurer *AVD Manager* (Créer un émulateur de téléphone, il ne doit pas y avoir d'espace dans le nom)

Référence : <https://reactnative.dev/docs/environment-setup>

Créer un nouveau projet

- Créer un répertoire de travail
`mkdir Worspace`
- Créer un répertoire contenant l'application *React Native*
`cd Workspace`
`npx react-native init MyReactNative`

Tester le nouveau projet

- Démarrer le simulateur de téléphone Android
- Démarrer *React Native*
`cd MyReactNative`
`npx react-native start`
- Ouvrir un nouveau terminal
- Démarrer l'application
`npx react-native run-android`

Structure d'un projet React Native

Fichier/répertoire	Description
node_modules	Répertoire contenant les modules Node.js. Il ne faut jamais modifier le contenu de ce répertoire.
app.json	Fichier qui permet de configurer l'application.
package.json	Fichier qui permet de gérer les dépendances.
App.js	Fichier qui est exécuté au lancement de l'application.
assets	Répertoire contenant les fichiers en lien avec l'application comme les images, les icones...
components	Répertoire contenant les composants de l'application.

Les composants

Un composant est un élément graphique affiché par l'interface utilisateur. Toutes les vues *React Native* sont composées entièrement de composants. Les éléments graphiques, vues, boutons, textes, images, listes, etc., sont aussi des composants. Cette notion de composants permet de découper nos interfaces utilisateur en pièces indépendantes et réutilisables, et de penser à chaque pièce séparément.

Exemple 1 : Composant simple

Créer un le fichier `Search.js` dans le répertoire `components`

```
1 // components/Search.js
2
3 import React from 'react'
4 import { View, TextInput, Button } from 'react-native'
5
6 class Search extends React.Component {
7   render() {
8     return (
9       <View>
10         <TextInput placeholder='Titre du film' />
11         <Button title='Rechercher' onPress={() => { }} />
12       </View>
13     )
14   }
15 }
16
```

```
17 export default Search
```

```
1 // App.js
2
3 import React from 'react'
4 import Search from '../components/Search'
5
6 export default class App extends React.Component {
7   render() {
8     return (
9       <Search />
10    )
11  }
12 }
```

Le composant Search affiche une vue avec un champ de saisie de texte et un bouton. Les 3 éléments utilisés sont :

- View : une vue.
- TextInput : un champ de saisie de texte.
- Button : un bouton.

Les composants doivent être exportés afin d'être utilisé par l'application.

Mise en forme d'un composant

Tous les composants *React Native* possèdent une propriété `style`. Cette propriété permet, d'appliquer des styles. Parmi les styles que l'on applique il y a :

- La couleur;
- La taille de la police;
- Les marges;
- L'alignement d'éléments entre eux;
- L'ajout de bordures;
- Etc.

Dans l'exemple suivant la propriété `style` du composant `<TextInput>` sera modifié.

Exemple 2 : Mise en forme du composant <TextInput>

```
1 // components/SearchStyle.js
2
3 import React from 'react'
4 import { StyleSheet, View, TextInput, Button } from 'react-native'
5
6 class SearchStyle extends React.Component {
7   render() {
8     return (
9       <View>
10         <TextInput style={styles.textinput} placeholder=
11           'Titre du film' />
12         <Button title='Rechercher' onPress={() => { }} />
13       </View>
14     )
15   }
16 }
17
18 const styles = StyleSheet.create({
19   textinput: {
20     marginLeft: 5,
21     marginRight: 5,
22     height: 50,
23     borderColor: '#000000',
24     borderWidth: 1,
25     paddingLeft: 5
26   }
27 })
28
29 export default SearchStyle
```

StyleSheet est une classe de *React Native* qui permet d'augmenter les performances des applications :

- Sans StyleSheet, à chaque fois que l'application appelle un style, un nouvel objet style est créé.
- Avec StyleSheet, un style est associé à un identifiant. Chaque fois que l'application appelle le style, StyleSheet utilise l'identifiant pour récupérer l'objet avec le style.

Mise en forme avec Flex Box

En *React Native*, tous les composants utilisent Flex Box (*Flexible Box Layout Module*). Les Flex Box sont un standard CSS3 de disposition des éléments dans une page Web. Ce standard permet d'avoir un design adaptatif en fonction de la taille de l'écran d'affichage.

Exemple 3 : Dimensions statiques

```
1 return (
```

```

2   <View style={{ backgroundColor: 'yellow' }}>
3     <View style={{ height: 335, backgroundColor: 'red' }}></View>
4     <View style={{ height: 335, backgroundColor: 'green' }}></View>
5   </View>
6 );

```

Exemple 4 : Moitié-moitié

```

1   return (
2     <View style={{ flex: 1, backgroundColor: 'yellow' }}>
3       <View style={{ flex: 1, backgroundColor: 'red' }}></View>
4       <View style={{ flex: 1, backgroundColor: 'green' }}></View>
5     </View>
6   )

```

La vue rouge de prendre une part (flex : 1) et à la vue verte prend également une part (flex : 1). Étant donné qu'elles possèdent des parts égales, elles vont se partager l'écran.

Exemple 5 : Un tiers deux tiers

```

1   return (
2     <View style={{ flex: 1, backgroundColor: 'yellow' }}>
3       <View style={{ flex: 1, backgroundColor: 'red' }}></View>
4       <View style={{ flex: 2, backgroundColor: 'green' }}></View>
5     </View>
6   )

```

Exemple 6 : Alignement horizontal

```

1   return (
2     <View style={{ flex: 1, flexDirection: 'row', backgroundColor: 'yellow' }}>
3       <View style={{ flex: 1, backgroundColor: 'red' }}></View>
4       <View style={{ flex: 2, backgroundColor: 'green' }}></View>
5       <View style={{ flex: 3, backgroundColor: 'blue' }}></View>
6     </View>
7   )

```

Par défaut, les composants possèdent un style `flexDirection: 'column'` qui aligne verticalement les composants entre eux. S'il est nécessaire de modifier la direction de l'alignement entre des composants, il faut modifier le style `flexDirection` du composant parent. C'est lui qui va dire à ses composants enfants comment s'aligner. Le style `flexDirection` a deux valeurs possibles :

- `column` : alignement vertical.
- `row` : alignement horizontal.

Exemple 7 : Trois petites boites

```
1 return (  
2   <View style={{ flex: 1, backgroundColor: 'yellow' }}>  
3     <View style={{ height: 50, width: 50, backgroundColor: 'red' }}></View>  
4     <View style={{ height: 50, width: 50, backgroundColor: 'green' }}></View>  
5     <View style={{ height: 50, width: 50, backgroundColor: 'blue' }}></View>  
6   </View>  
7 )
```

Exemple 8 : Centrer verticalement (sur l'axe principal)

```
1 return (  
2   <View style={{ flex: 1, justifyContent: 'center', backgroundColor: 'yellow' }}>  
3     <View style={{ height: 50, width: 50, backgroundColor: 'red' }}></View>  
4     <View style={{ height: 50, width: 50, backgroundColor: 'green' }}></View>  
5     <View style={{ height: 50, width: 50, backgroundColor: 'blue' }}></View>  
6   </View>  
7 )
```

Le style `justifyContent` permet de définir comment sont distribués les composants enfants sur l'axe principal. L'axe principal dépend de l'alignement choisi avec le style `flexDirection`:

- `flexDirection: 'column'`, l'alignement est vertical et l'axe principal est Y. L'axe secondaire est X.
- `flexDirection: 'row'`, l'alignement est horizontal et l'axe principal est X. L'axe secondaire est Y.

Il existe 6 valeurs possibles pour le style `justifyContent`:

Style	Description
center	Aligner les éléments au centre.
flex-start	Aligner les éléments au début.
flex-end	Aligner les éléments à la fin
space-between	Les éléments s'affichent sur tout l'axe et laissent de l'espace entre eux.
space-around	Les éléments s'affichent sur tout l'axe et laissent de l'espace entre eux. L'espace aux extrémités est 2 fois inférieur à l'espace entre les éléments.
space-evenly	Les éléments s'affichent sur tout l'axe et laissent de l'espace entre eux. L'espace aux extrémités est le même que celui entre les

	éléments.
--	-----------

Exemple 9 : Centrer horizontal (sur l'axe secondaire)

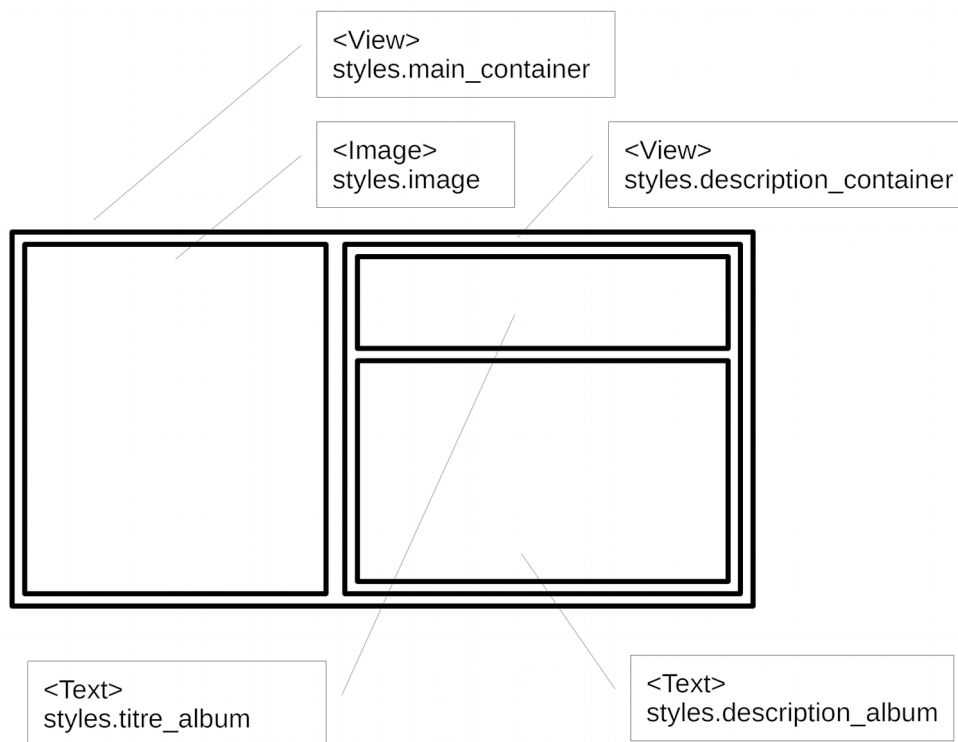
```

1 return (
2   <View style={{ flex: 1, alignItems: 'center', backgroundColor: 'yellow' }}>
3     <View style={{ height: 50, width: 50, backgroundColor: 'red' }}></View>
4     <View style={{ height: 50, width: 50, backgroundColor: 'green' }}></View>
5     <View style={{ height: 50, width: 50, backgroundColor: 'blue' }}></View>
6   </View>
7 )

```

Le style `alignItems` est comme `justifyContent`, sauf qu'il s'applique sur l'axe secondaire.

Exemple 10 : Composant permettant d'afficher la description d'un CD



```

1 return (
2   <View style={styles.main_container}>
3     <Image
4       style={styles.image}
5       source={require('./asset/DavidGilmour.jpg')}
6     />
7     <View style={styles.content_container}>
8       <Text style={styles.titre_album}>On the island</Text>
9       <Text style={styles.description_album}>Artiste: David Gilmour</Text>

```



```

10     </View>
11 </View>
12 )

```

```

1  const styles = StyleSheet.create({
2    main_container: {
3      height: 190,
4      flexDirection: 'row',
5      borderColor: 'red',
6      borderWidth: 1
7    },
8    image: {
9      width: 180,
10     height: 180,
11     margin: 5,
12     borderColor: 'gray',
13     borderWidth: 1
14   },
15   content_container: {
16     flex: 1,
17     margin: 5,
18     borderColor: 'red',
19     borderWidth: 1
20   },
21   titre_album:{
22     fontWeight: 'bold',
23     fontSize: 20,
24     flex: 1,
25     flexWrap: 'wrap',
26     paddingLeft: 5,
27     borderColor: 'gray',
28     borderWidth: 1
29   },
30   description_album: {
31     flex: 5,
32     fontStyle: 'italic',
33     color: 'black',
34     paddingLeft: 5
35   }
36 }
37 );

```

Propriétés d'un composant

Exemple 11 : Utilisation des propriétés

```

1  // components/Bouton.js
2

```

```

3 import React from 'react'
4 import { View, Button } from 'react-native'
5
6 class Bouton extends React.Component {
7   render() {
8     console.log(this.props)
9     return (
10       <View>
11         <Button title={this.props.titre} onPress={() =>
12           {this.props.onPress()}} />
13       </View>
14     )
15   }
16 }
17
18 export default Bouton;

```

```

1 // App.js
2
3 import React from 'react'
4 import Bouton from './components/Bouton'
5
6 export default class App extends React.Component {
7   render() {
8     return(
9       <Bouton titre='Allo' onPress={()=>affiche('1')} />
10     )
11   }
12 }
13
14 function affiche(nombre){
15   console.log(nombre);
16 }

```

Propriété onPress

Exemple 12 : Utilisation de la propriété onPress avec un bouton personnalisé

```

1 // Components/BoutonPers.js
2
3 import React from 'react'
4 import { StyleSheet, TouchableOpacity, Text } from 'react-native'
5
6 class BoutonPers extends React.Component {
7   render() {
8     console.log(this.props.texte);
9     return (
10       <TouchableOpacity style={styles.touche} onPress={() =>
11         {this.props.onPress(this.props.texte);}}>

```

```

12         <Text style={styles.texte}>{this.props.texte}</Text>
13     </TouchableOpacity>
14     )
15   }
16 }
17
18 const styles = StyleSheet.create({
19   touche: {
20     margin: 5,
21     padding: 10,
22     alignItems: "center",
23     justifyContent: "center",
24     borderRadius: 10,
25     backgroundColor: 'blue'
26   },
27   texte: {
28     fontSize: 40,
29     color: "white",
30     fontWeight: "bold"
31   }
32 });
33
34 export default BoutonPers

```

L'apparence du composant <Button/> est difficilement modifiable. Lorsqu'il est nécessaire d'avoir un bouton personnalisé il faut utiliser le composant <TouchableOpacity/> (ligne 10). Ce composant permet de créer une zone tactile à l'écran l'appareil mobile.

```

1  // App.js
2
3  import React from 'react'
4  import Search from '../Components/Search'
5  import BoutonPers from '../Components/BoutonPers'
6
7  import { StyleSheet, View } from 'react-native'
8
9  export default class App extends React.Component {
10
11    handleOnPress = (texte) => {
12      alert(texte);
13    }
14
15    render() {
16
17      return (
18        <View style={styles.wrapper}>
19          <BoutonPers texte="bouton 1" onPress={(touche) =>
20            { this.handleOnPress(touche) }} />
21          <BoutonPers texte="bouton 2" onPress={(touche) =>

```

```

22         { this.handleOnPress(touche) }} />
23     </View>
24
25     )
26   }
27 }
28
29 const styles = StyleSheet.create({
30   wrapper: {
31     flex: 1
32   }
33 });

```

États d'un composant

Exemple 13 : Utilisation des états

```

1  // components/SearchAction.js
2
3  import React from 'react'
4  import { View, TextInput, Button } from 'react-native'
5
6  class SearchAction extends React.Component {
7    state = {
8      TitreFilm: ''
9    }
10
11    handleTitreFilm = (texte) => {
12      this.setState({ TitreFilm: texte })
13    }
14
15    afficher = (TitreFilm) => {
16      alert(TitreFilm)
17    }
18    render() {
19      return (
20        <View>
21          <TextInput placeholder='Titre du film' onChangeText =
22                                {this.handleTitreFilm}/>
23          <Button title='Rechercher' onPress={() =>
24                                {this.afficher(this.state.TitreFilm)}} />
25        </View>
26      )
27    }
28  }
29
30  export default SearchAction

```

Pour modifier l'état d'un composant, il faut toujours utiliser la méthode `setState()` (ligne 12). Une fois les états modifiés par la méthode `setState()` celle-ci exécute la méthode `render()` afin d'afficher, si nécessaire, les modifications de l'état.

```
1 // App.js
2
3 import React from 'react'
4 import SearchAction from './components/SearchAction'
5
6 import { View } from 'react-native'
7
8 export default class App extends React.Component {
9   render() {
10     return(
11       <SearchAction />
12     )
13   }
14 }
```

Communication enfant-parent (*lifting state up*)

Plusieurs composants ont souvent besoin de refléter les mêmes données de façon dynamique. Il est donc possible de faire remonter l'état du composant enfant vers le composant parent.

Exemple 14 : Transfert de l'état d'un composant au composant parent

```
1 // components/SearchAction.js
2
3 import React from 'react'
4 import { View, TextInput, Button } from 'react-native'
5
6 class SearchAction extends React.Component {
7   state = {
8     TitreFilm: ''
9   }
10
11   handleTitreFilm = (texte) => {
12     this.setState({ TitreFilm: texte });
13     console.log(texte);
14   }
15
16   afficher = (TitreFilm) => {
17     alert(TitreFilm);
18     this.props.onLeTexteChange(TitreFilm);
19   }
20 }
```

```

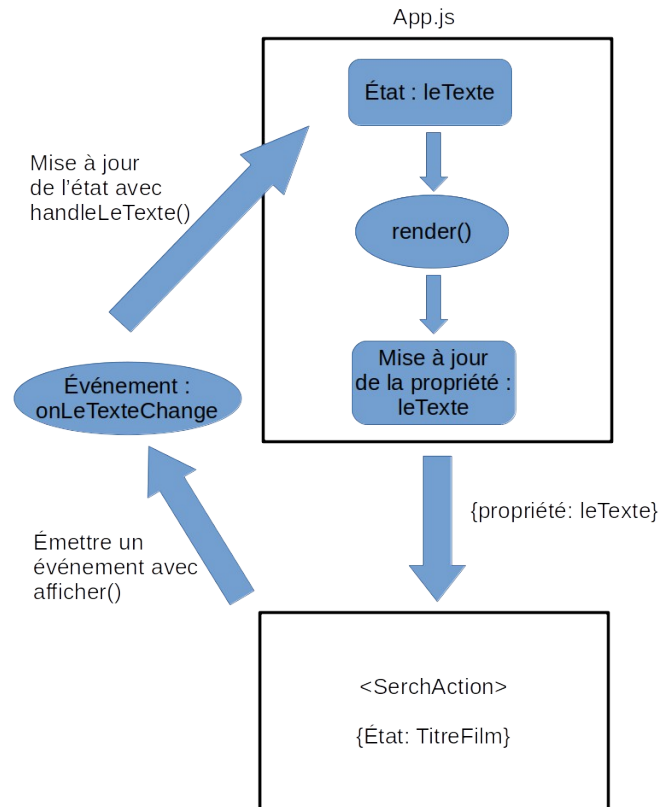
20     render() {
21         return (
22             <View>
23                 <TextInput placeholder='Titre du film'
24                     onChangeText = {this.handleTitreFilm}/>
25                 <Button title='Rechercher'
26                     onPress={() => {this.afficher(this.state.TitreFilm);}} />
27             </View>
28         )
29     }
30 }
31
32 export default SearchAction

```

```

1  // App.js
2
3  import React from 'react'
4  import SearchStateUp from '../components/SearchStateUp'
5
6  import { View } from 'react-native'
7
8  export default class App extends React.Component {
9      state = {
10         leTexte: ''
11     }
12
13     handleLeTexte = (texte) =>{
14         this.setState({ leTexte: texte });
15         console.log("allo " + texte);
16     }
17
18     render() {
19         return(
20             <SearchStateUp
21                 leTexte={this.state.leTexte}
22                 onLeTexteChange={this.handleLeTexte}
23             />
24         )
25     }
26 }

```



Exemple complet

Dans cette section une application qui affiche la météo actuelle est réalisée.

1. Créer un répertoire contenant la nouvelle application

```
npx react-native init MeteoApp
```

2. Démarrer le simulateur de téléphone Android

3. Démarrer *React Native*

```
cd MeteoApp
```

```
npx react-native start
```

4. Ouvrir un nouveau terminal

5. Démarrer l'application

```
npx react-native run-android
```

6. Créer un répertoire `components` et à l'intérieur un fichier `Entete.js`. Copier dans le fichier `Entete.js` le contenu de l'exemple ci-bas.
7. Remplacer le contenu `App.js` par le contenu du fichier `App.js` de l'exemple 1.
8. Dans le fichier `App.js`, remplacer la balise `<Search/>` par la balise `<Entete/>`. Ne pas oublier d'importer le composant `Entete.js`. Vérifier le fonctionnement de l'application.
9. Dans le fichier `App.js`, ajouter les balises `<SafeAreaView/>` et `<StatusBar/>`. Créer le style `main_container`.
10. Créer le composant `PiedDePage.js` et l'intégrer dans l'application.
11. Finalement, créer le composant `Meteo.js` et l'intégrer dans l'application.

Exemple 15 : Réaliser une application qui affiche la météo actuelle

```
1  // components/Entete.js
2
3  import React from 'react'
4  import { StyleSheet, View, Text } from 'react-native'
5
6  class Entete extends React.Component {
7    render() {
8      return (
9        <View style={styles.entete}>
10          <Text style={styles.titre}>MaMétéo</Text>
11        </View>
12      )
13    }
14  }
15
16  const styles = StyleSheet.create({
17    entete: {
18      height: 50,
19      backgroundColor: 'red',
20      justifyContent: 'center'
21    },
22    titre: {
23      textAlignVertical: "center",
24      textAlign: "center",
25      fontSize: 20
26    }
27  })
28
29  export default Entete
```



```

1 // components/Meteo.js
2
3 import React from 'react'
4 import { StyleSheet, View, Text, Image } from 'react-native'
5
6 class Meteo extends React.Component {
7   render() {
8     return (
9       <View style={styles.meteo}>
10         <Image
11           style={styles.icone}
12           source={this.props.icone}
13         />
14         <Text>{this.props.description}</Text>
15         <Text style={styles.temperature}>{this.props.temperature}
16                                           &deg;C</Text>
17       </View>
18     )
19   }
20 }
21
22 const styles = StyleSheet.create({
23   meteo: {
24     flex: 1,
25     alignItems: 'center'
26   },
27   icone: {
28     marginTop: 40
29   },
30   temperature: {
31     fontSize: 40,
32     fontWeight: "bold"
33   }
34 })
35
36 export default Meteo

```

```

1 // components/PiedDePage.js
2
3 import React from 'react'
4 import { StyleSheet, View, Button } from 'react-native'
5
6 class PiedDePage extends React.Component {
7   render() {
8     return (
9       <View style={styles.piedDePage}>
10         <Button title="Charger météo"
11               onPress={() => { this.props.onPress() }} />
12       </View>
13     )
14   }
15 }
16

```

```

17 const styles = StyleSheet.create({
18   piedDePage: {
19     position: 'absolute',
20     left: 0,
21     right: 0,
22     bottom: 0
23   }
24 })
25
26 export default PiedDePage

```

```

1  import React from 'react';
2  import { StyleSheet, SafeAreaView, StatusBar } from 'react-native';
3  import Entete from '../componants/Entete';
4  import Meteo from '../componants/Meteo';
5  import PiedDePage from '../componants/PiedDePage';
6  import { API_KEY } from '../assets/WeatherAPIKey';
7
8  export default class App extends React.Component {
9    state = {
10      temperature: "---",
11      description: " ",
12      icone: this.getIcône('unknown')
13    }
14
15    componentDidMount() {
16      console.log("je suis prêt");
17      this.fetchMeteo();
18    }
19
20    render() {
21      return (
22        <SafeAreaView style={styles.main_container}>
23          <StatusBar />
24          <Entete />
25          <Meteo
26            temperature={this.state.temperature}
27            description={this.state.description}
28            icone={this.state.icone}
29          />
30          <PiedDePage onPress={() => {this.fetchMeteo();
31                                console.log("je fetch");}} />
32        </SafeAreaView>
33      )
34    }
35
36    fetchMeteo() {
37      const URL = "http://api.openweathermap.org/data/2.5/weather?
38                  q=Montreal,ca&lang=fr&units=metric&appid=" + API_KEY;
39
40      fetch(URL)
41        .then(res => res.json())
42        .then(json => {

```

```

43         this.setState({
44             temperature: json.main.temp,
45             description: json.weather[0].description,
46             icone: this.getIcône(json.weather[0].icon)
47         });
48     });
49 }
50
51 getIcône(nom) {
52     switch (nom) {
53         case 'unknown':
54             return (require('./assets/unknown.png'));
55         case '01d':
56             return (require('./assets/01d.png'));
57         case '01n':
58             return (require('./assets/01n.png'));
59         case '02d':
60             return (require('./assets/02d.png'));
61         case '02n':
62             return (require('./assets/02n.png'));
63         case '03d':
64             return (require('./assets/03d.png'));
65         case '03n':
66             return (require('./assets/03n.png'));
67         case '04d':
68             return (require('./assets/04d.png'));
69         case '04n':
70             return (require('./assets/04n.png'));
71         case '09d':
72             return (require('./assets/09d.png'));
73         case '09n':
74             return (require('./assets/09n.png'));
75         case '10d':
76             return (require('./assets/10d.png'));
77         case '10n':
78             return (require('./assets/10n.png'));
79         case '11d':
80             return (require('./assets/11d.png'));
81         case '11n':
82             return (require('./assets/11n.png'));
83         case '13d':
84             return (require('./assets/13d.png'));
85         case '13n':
86             return (require('./assets/13n.png'));
87         case '50d':
88             return (require('./assets/50d.png'));
89         case '50n':
90             return (require('./assets/50n.png'));
91         default:
92             return (require('./assets/unknown.png'));
93     }
94 }
95 }
96
97 const styles = StyleSheet.create({

```

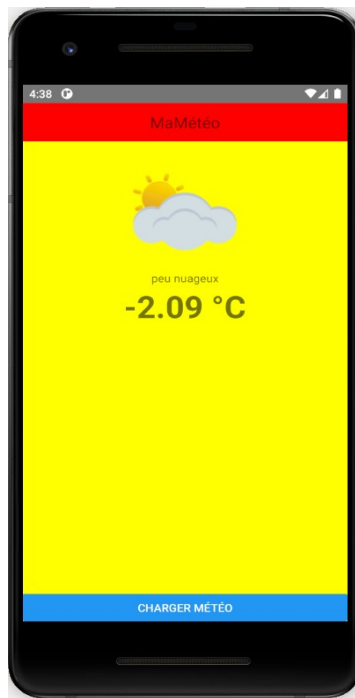
```

98   main_container: {
99     flex: 1,
100    backgroundColor: 'yellow'
101  }
102 })

```

La méthode `componentDidMount()` (ligne 15) est appelée immédiatement après que le composant a été inséré dans le DOM. C'est dans cette méthode qu'il faut habituellement charger des données via une requête réseau. Il est possible d'utiliser la méthode `setState()` directement dans `componentDidMount()`.

Pour être capable de modifier la source de la balise `<Image/>` en cours d'exécution de l'application il est nécessaire de créer la méthode `getIcône()`. Cette méthode permet de charger en mémoire l'image qui sera utilisé par la balise `<Image/>`.



Navigation entre les vues

En développement mobile, la navigation représente le passage d'une vue à une autre. La navigation n'est pas la même entre iOS et Android :

- Sous **iOS**, on pousse nos vues : une animation affiche nos nouvelles vues par un *slide* de droite à gauche.
- Sous **Android**, on présente nos vues : une animation affiche nos nouvelles vues par un *slide* de bas en haut.

React Native ne propose pas de solutions pour respecter cette différence dans les comportements natifs. La communauté *React Native* a créé une librairie pour gérer la différence entre la navigation sous iOS et Android. La librairie *React Navigation* est donc devenue un incontournable du développement en *React Native*.

Voici les différents type de navigation sous *React Navigation* :

- ***StackNavigator*** : c'est la navigation la plus basique où on pousse une vue sur iOS et présente une vue sur Android. Le *StackNavigator* gère une pile de vues qui augmente lorsque vous naviguez vers une nouvelle vue et diminue lorsque vous revenez en arrière. C'est cette navigation que l'on va utiliser dans ce chapitre.
- ***TabNavigator*** : permet de créer une barre d'onglets, en haut ou en bas, de votre application. On utilisera ce type de navigation un peu plus loin dans ce cours, lorsque l'on voudra couper notre application en plusieurs onglets.
- ***DrawerNavigator*** : permet de créer un menu dit « hamburger », à gauche de nos vues, avec une liste d'entrées pour chacune de nos vues.

Installer React Navigation

1. Créer un nouveau projet

```
npx react-native init testStackNav
```

2. Installer *React Navigation* dans le nouveau projet

```
cd testStackNav
```

```
npm install @react-navigation/native
```

```
npm install react-native-screens react-native-safe-area-context
```

```
npm install @react-navigation/native-stack
```

3. Démarrer le simulateur de téléphone Android

4. Démarrer React Native

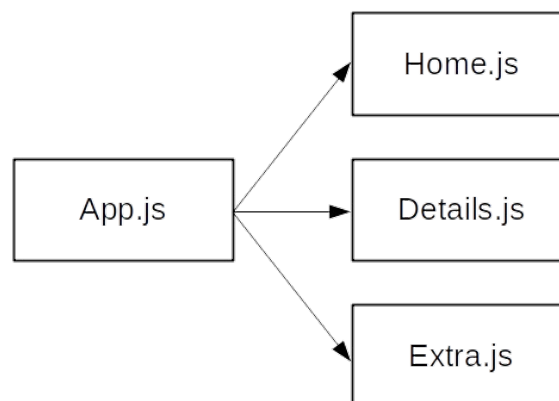
```
npx react-native start
```

5. Ouvrir un nouveau terminal

6. Démarrer l'application

```
npx react-native run-android
```

Navigation par pile (*StackNavigator*)



```
1 // App.js
2
3 import * as React from 'react';
4 import { View, Text } from 'react-native';
5 import { NavigationContainer } from '@react-navigation/native';
6 import { createNativeStackNavigator } from '@react-navigation/native-stack';
7 import Home from './components/Home';
8 import Details from './components/Details';
9 import Extras from './components/Extras';
10
11 function HomeScreen() {
12   return (
13     <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
14       <Text>Home Screen</Text>
15     </View>
16   );
17 }
```

```

18
19 const Stack = createNativeStackNavigator();
20
21 function App() {
22   return (
23     <NavigationContainer>
24       <Stack.Navigator>
25         <Stack.Screen name="Home" component={Home} />
26         <Stack.Screen name="Details" component={Details} />
27         <Stack.Screen name="Extras" component={Extras} />
28       </Stack.Navigator>
29     </NavigationContainer>
30   );
31 }
32
33 export default App;

```

Les routes sont définies aux lignes 25 à 27. Il y a 3 routes : Home, Details et Extras.

```

1 // components/Home.js
2
3 import React from 'react'
4 import { StyleSheet, View, Text, Button } from 'react-native'
5
6 class Home extends React.Component {
7
8   static navigationOptions = {
9     title: 'Home',
10   };
11
12   render() {
13     return (
14       <View style={styles.wrapper}>
15         <Text>Page d'accueil</Text>
16         <Button
17           title="Allez aux détails"
18           onPress={()=>this.props.navigation.navigate('Details')}
19         />
20         <Button
21           title="Allez aux Extras"
22           onPress={()=>this.props.navigation.navigate('Extras',{id:1})}
23         />
24       </View>
25     )
26   }
27 }
28
29 const styles = StyleSheet.create({
30   wrapper: {
31     flex: 1,
32     alignItems: 'center',
33     justifyContent: 'center',
34   }
35 });
36

```

```
37 export default Home
```

L'attribut statique `navigationOptions` permet de configurer les routes.

```
1 // components/Details.js
2
3 import React from 'react'
4 import { StyleSheet, View, Text } from 'react-native'
5
6 class Details extends React.Component {
7
8     static navigationOptions = {
9         title: 'Details',
10        headerStyle: {
11            backgroundColor: '#f4511e',
12        }
13    };
14
15    render() {
16        return (
17            <View style={styles.wrapper}>
18                <Text>Les détails</Text>
19            </View>
20        )
21    }
22 }
23
24 const styles = StyleSheet.create({
25     wrapper: {
26         flex: 1,
27         alignItems: 'center',
28         justifyContent: 'center'
29     }
30 });
31
32 export default Details
```

Passage de paramètres à une route

```
1 // components/Home.js
2
3 ...
4 ...
5     <Button
6         title="Allez aux Extras"
7         onPress={() => this.props.navigation.navigate('Extras', { id: 1 })}
8     />
9 ...
10 ...
```


Ligne 7, le paramètre id est passé à la route Extras.

```
1 // components/Extras.js
2
3 import React from 'react'
4 import { StyleSheet, View, Text, Button } from 'react-native'
5
6 class Extras extends React.Component {
7   static navigationOptions = {
8     headerTitle: "allo",
9     headerRight: () => (
10       <Button
11         onPress={() => alert('Ceci est un bouton!')}
12         title="Info"
13       />
14     ),
15   };
16
17   render() {
18     console.log("props ->", this.props);
19     const { navigation, route } = this.props;
20     console.log("param ->", route.params.id);
21     return (
22       <View style={styles.wrapper}>
23         <Text>Les extras</Text>
24       </View>
25     )
26   }
27 }
28
29 const styles = StyleSheet.create({
30   wrapper: {
31     flex: 1,
32     alignItems: 'center',
33     justifyContent: 'center'
34   }
35 });
36
37 export default Extras
```

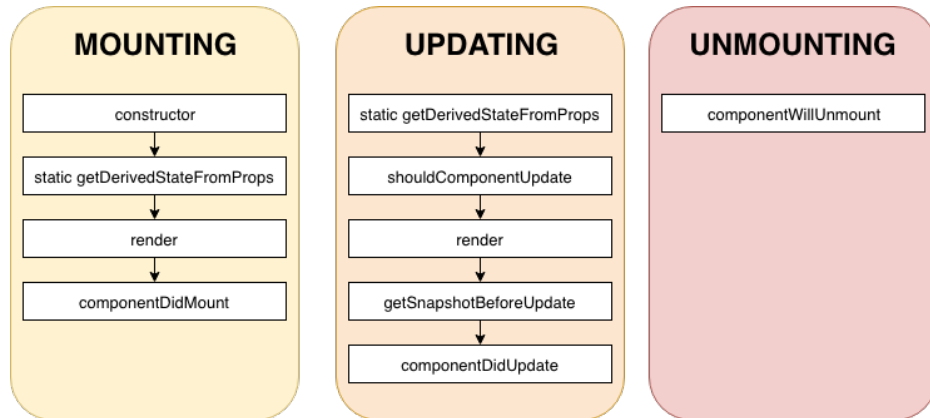
Les paramètres sont récupérés via l'attribut `route.params` (ligne 20).

Cycle de vie des composants

Un cycle de vie d'un composant correspond à un changement de son état. Il existe 3 cycles de vie : montage, mise à jour et démontage.

- **Montage** (*mounting*) : Affichage d'un composant à l'écran. Ce cycle de vie englobe l'initialisation du composant jusqu'à son affichage à l'écran.

- **Mise à jour** (updating) : Mise à jour du composant. Après la mise à jour des états du composant celui-ci est ré-affiché.
- **Démontage** (unmounting) : Suppression d'un composant de l'écran.



Prenez l'exemple du cycle de vie montage. Lorsque qu'un composant est ajouté à l'écran, voici ce qu'il se passe :

1. Le constructeur (*constructor*) du composant est appelé.
2. La méthode statique `getDerivedStateFromProps()` est exécutée. À cette étape, il est possible de modifier l'état d'un composant avant qu'il ne soit affiché.
3. La méthode `render()` affiche le composant.
4. La méthode `componentDidMount()` est appelée quand l'affichage du composant est terminé.

Il est toujours possible de surcharger le constructeur ou une des méthodes du cycle de vie d'un composant.

Exemple 16 : Surcharger la méthode `componentDidMount()`

```

1  ...
2  ...
3  componentDidMount() {
4      console.log("je suis prêt");
5      this.fetchMeteo();
6  }
7  ...
8  ...

```

Navigation par menu (*Drawernavigator*)

1. Installer *React Navigation* dans le nouveau projet

```
npm install @react-navigation/native
```

```
npm install react-native-screens react-native-safe-area-context
```

```
npm install @react-navigation/drawer
```

```
npm install react-native-gesture-handler react-native-reanimated
```

2. Modifier le fichier `android/app/build.gradle` en modifiant la ligne suivante :

```
1 project.ext.react = [enableHermes: false,] // update here true
```

3. Modifier le fichier `android/app/src/main/java[Your Packages]/MainApplication.java` en ajoutant les lignes suivantes :

```
1 import com.facebook.react.bridge.JSIModulePackage;
2     import com.swmansion.reanimated.ReanimatedJSIModulePackage;
3
4 private final ReactNativeHost mReactNativeHost =
5     new ReactNativeHost(this) {
6     ...
7     @Override
8     protected JSIModulePackage getJSIModulePackage() {
9         return new ReanimatedJSIModulePackage();
10    }
11 };
```

4. Modifier le fichier `babel.config.js` en ajoutant les lignes suivantes :

```
1 module.exports = {
2     ...
3     plugins: ["react-native-reanimated/plugin"], //add this line
4 };
```

5. Modifier le fichier `index.js` en ajoutant la ligne suivante :

```
1 import 'react-native-gesture-handler'
```

6. Démarrer l'application une première fois avec l'instruction suivante :

```
npmx react-native start --reset-cache
```

Référence : <https://stackoverflow.com/questions/70968067/typeerror-undefined-is-not-an-object-evaluating-innervativemodule-installcore>

```
1 // App.js
2
3 import * as React from 'react';
4 import { Button, View } from 'react-native';
```

```

5 import { createDrawerNavigator } from '@react-navigation/drawer';
6 import { NavigationContainer } from '@react-navigation/native';
7 import Home from './components/Home'
8 import Details from './components/Details'
9 import Extras from './components/Extras'
10
11
12 function NotificationsScreen({ navigation }) {
13   return (
14     <View style={{ flex:1, alignItems:'center', justifyContent:'center' }}>
15       <Button onPress={() => navigation.goBack()} title="Go back home" />
16     </View>
17   );
18 }
19
20 const Drawer = createDrawerNavigator();
21
22 export default function App() {
23   return (
24     <NavigationContainer>
25       <Drawer.Navigator initialRouteName="Home">
26         <Drawer.Screen name="Home" component={Home} />
27         <Drawer.Screen name="Détails" component={Details} />
28         <Drawer.Screen name="Extras" component={Extras} />
29       </Drawer.Navigator>
30     </NavigationContainer>
31   );
32 }

```

```

1 // components/Home.js
2
3 import React from 'react';
4 import { StyleSheet, View, Text, Button } from 'react-native';
5
6 class Home extends React.Component {
7
8   static navigationOptions = {
9     title: 'Home',
10    headerLeft: 'Home',
11
12  };
13
14  render() {
15    return (
16      <View style={styles.wrapper} >
17        <Text>Page d'accueil</Text>
18        <Button
19          title="Ouvrir Menu"
20          onPress={()=>this.props.navigation.openDrawer()}
21        />
22      </View>
23    )
24  }

```

```

25 }
26
27 const styles = StyleSheet.create({
28   wrapper: {
29     flex: 1,
30     alignItems: 'center',
31     justifyContent: 'center'
32   }
33 });
34
35 export default Home
36 }

```

Navigation par onglet (*TabNavigator*)

1. Installer *React Navigation* dans le nouveau projet

```
npm install @react-navigation/native
```

```
npm install react-native-screens react-native-safe-area-context
```

```
npm install @react-navigation/bottom-tabs
```

```
npm install @react-navigation/material-bottom-tabs react-native-paper
react-native-vector-icons
```

2. Modifier le fichier `android/app/build.gradle` en ajoutant la ligne suivante :

```
1 apply from: "../../node_modules/react-native-vector-icons/fonts.gradle"
```

Référence :

- <https://reactnavigation.org/docs/tab-based-navigation>
- <https://github.com/oblador/react-native-vector-icons#installation>

```

1 import * as React from 'react';
2 import { Text, View } from 'react-native';
3 import { NavigationContainer } from '@react-navigation/native';
4 import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
5 import MaterialCommunityIcons from
'react-native-vector-icons/MaterialCommunityIcons';
6
7 function HomeScreen() {
8   return (
9     <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
10       <Text>Home!</Text>

```

```

11     </View>
12   );
13 }
14
15 function SettingsScreen() {
16   return (
17     <View style={{ flex:1, justifyContent:'center', alignItems:'center' }}>
18       <Text>Settings!</Text>
19     </View>
20   );
21 }
22
23 const Tab = createBottomTabNavigator();
24
25 export default function App() {
26   return (
27     <NavigationContainer>
28       <Tab.Navigator
29         screenOptions={({ route }) => ({
30           tabBarIcon: ({ color, size }) => {
31             const icons = {
32               Home: 'home',
33               Settings: 'account',
34             };
35
36             return (
37               <MaterialCommunityIcons
38                 name={icons[route.name]}
39                 color={color}
40                 size={size}
41               />
42             );
43           },
44         )}}
45       >
46         <Tab.Screen name="Home" component={HomeScreen} />
47         <Tab.Screen name="Settings" component={SettingsScreen} />
48       </Tab.Navigator>
49     </NavigationContainer>
50   );
51 }

```

Persistence des données

La plupart des applications mobiles ont besoins de conserver les données des utilisateurs entre les sessions. Pour ce faire *React Native* fourni le module *AsyncStorage* qui permet de créer un stockage local persistant.

Installer *react-native-async-storage* dans le nouveau projet

```
npm install @react-native-async-storage/async-storage
```

```
1 // components/StockageLocal.js
2
3 import React from 'react'
4 import { StyleSheet, View, Text, TextInput, Button } from 'react-native'
5 import AsyncStorage from '@react-native-async-storage/async-storage';
6
7 class StockageLocal extends React.Component {
8   static navigationOptions = {
9     headerTitle: "Async Storage",
10   };
11
12   state = {
13     nom: '',
14     maBdLocale: {
15       nom: ''
16     }
17   }
18
19   componentDidMount() {
20     this.obtenirNom();
21   }
22
23   modifierNom = (texte) => {
24     this.setState({ nom: texte });
25   }
26
27   enregistrerNom = async () => {
28     //alert(JSON.stringify(this.state.maBdLocale));
29     console.log('Nom: ' + this.state.nom);
30     this.setState({ maBdLocale:{nom: this.state.nom}}, async () => {
31       try {
32         const jsonValue = JSON.stringify(this.state.maBdLocale);
33         console.log("enregistrer: " + jsonValue);
34         await AsyncStorage.setItem('@storage_Key', jsonValue);
35       } catch (e) {
36         alert("Erreur à l'enregistrement!");
37       }
38     });
39   }
40
41   obtenirNom = async () => {
42     try {
43       const jsonValue = await AsyncStorage.getItem('@storage_Key');
44       console.log(jsonValue);
45       if (jsonValue != null){
46         this.setState({maBdLocale: JSON.parse(jsonValue)});
47       }
48     }
49     else{
50       this.setState({maBdLocale: ''});
```

```

51     }
52   } catch (e) {
53     alert("Erreur de lecture!");
54   }
55 }
56
57
58 render() {
59   return (
60     <View style={styles.wrapper}>
61       <TextInput
62         placeholder='Nom'
63         style={styles.textInput}
64         onChangeText={this.modifierNom}
65       />
66       <Button
67         title='Modifier Nom'
68         onPress={() => { this.enregistrerNom() }}
69       />
70       <Text>
71         Nom: {this.state.maBdLocale.nom}
72       </Text>
73     </View>
74   )
75 }
76 }
77
78 const styles = StyleSheet.create({
79   wrapper: {
80     flex: 1,
81     alignItems: 'center',
82     justifyContent: 'center',
83   },
84   textInput: {
85     borderWidth: 1,
86     width: '50%'
87   }
88 });
89
90 export default StockageLocal

```

Géolocalisation

Installer *react-native-geolocation-service* dans le nouveau projet

```
npm install --save react-native-geolocation-service
```

Note : `--save` permet d'enregistrer la librairie dans les dépendances du fichier `package.json`. Ce fichier recense toutes vos librairies. En ayant ajouté la librairie à ce fichier, si quelqu'un récupère

vosre projet ou que vous l'utilisez sur un autre ordinateur, vous pourrez installer toutes vos librairies en faisant juste : `npm install`.

```
1 // components/Geoloc.js
2
3 import React from 'react'
4 import { StyleSheet, View, Text, Button } from 'react-native';
5 import Geolocation from 'react-native-geolocation-service';
6
7 class Geoloc extends React.Component {
8   state = {
9     latitude: 0,
10    longitude: 0
11  }
12
13  static navigationOptions = {
14    title: 'Géolocalisation',
15    headerTitle: "Géolocalisation",
16    headerStyle: {
17      backgroundColor: '#f4511e',
18    }
19  };
20
21  render() {
22    return (
23      <View style={styles.wrapper}>
24        <Text>Latitude: {this.state.latitude}; Longitude:
25                                     {this.state.longitude}</Text>
26        <Button title="Ma position" onPress={()=>this.getPosition()}/>
27      </View>
28    )
29  }
30
31  getPosition() {
32    Geolocation.getCurrentPosition(
33      (position) => {
34        console.log(position);
35        this.setState({
36          latitude: position.coords.latitude,
37          longitude: position.coords.longitude
38        });
39      },
40      (error) => {
41        console.log(error.code, error.message);
42      },
43      { enableHighAccuracy:true, timeout:15000, maximumAge:10000 }
44    );
45  }
46 }
47
48 const styles = StyleSheet.create({
49   wrapper: {
50     flex: 1,
51     alignItems: 'center',
```

```

52         justifyContent: 'center'
53     }
54 });
55
56 export default GeoLoc

```

Base de données locale (*mongoose like*)

```

1  import AsyncStorage from '@react-native-async-storage/async-storage';
2
3  class MongoDB {
4      constructor(dbName) {
5          this.dbName = dbName;
6          this.memory = { database: false };
7      }
8
9      async insert(data) {
10         await this.initDatabase();
11         // Trouver le max _id
12         if (this.memory.database.length == 0) {
13             data._id = 1;
14         }
15         else {
16             let max = 0;
17             this.memory.database.forEach((item) => {
18                 if (item._id > max)
19                     max = item._id;
20             });
21             data._id = max + 1;
22         }
23         this.memory.database.push(data);
24         await AsyncStorage.setItem(this.dbName,
25                                   JSON.stringify(this.memory.database));
26     }
27
28     async findAll() {
29         await this.initDatabase();
30         return (this.memory.database);
31     }
32
33     async find(query) {
34     }
35
36     async findOne(query) {
37     }
38
39     async update(data, query) {
40     }
41
42

```

```

43
44     }
45
46     async initDatabase() {
47         if (!this.memory.database) {
48             this.memory.database = await this.createDatabase();
49         }
50         else {
51             this.memory.database = await this.getDatabase();
52         }
53     }
54
55     async getDatabase() {
56         let jsonValue = await AsyncStorage.getItem(this.dbName);
57         if (jsonValue != null) {
58             return JSON.parse(jsonValue);
59         }
60         else {
61             return this.createDatabase();
62         }
63     }
64
65     async createDatabase() {
66         await AsyncStorage.setItem(this.dbName, JSON.stringify([]));
67         return this.getDatabase();
68     }
69
70     clear() {
71         this.memory.database = false;
72     }
73
74 }
75
76 export default MongogoDB

```

```

1 // components/Mongoose.js
2
3 import React from 'react';
4 import { StyleSheet, View, Text, TextInput, Button, FlatList }
5                                     from 'react-native';
6 import MongogoDB from './MongogoDB.js';
7
8
9 class Mongoose extends React.Component {
10     static navigationOptions = {
11         headerTitle: "Moongoose",
12     };
13
14     state = {
15         nom: '',
16         prenom: '',
17         maDB: [],
18     }

```

```

19
20 componentDidMount() {
21     this.db = new MongogoDB('maBD');
22     //this.db.clear();
23 }
24
25 async ajouter() {
26     await this.db.insert({nom:this.state.nom, prenom:this.state.prenom})
27     //await this.db.insert({ nom: "Gill", prenom: "Stephane" })
28     this.db.findAll().then((result) => {
29         //console.log(result);
30         this.setState({maBD: result});
31     });
32 }
33
34 renderRow(item) {
35     return (
36         <View style={styles.ligne}>
37             <View style={styles.cel}>
38                 <Text>{item.nom}</Text>
39             </View>
40             <View style={styles.cel}>
41                 <Text>{item.prenom}</Text>
42             </View>
43         </View>
44     );
45 }
46
47 render() {
48     return (
49         <View style={styles.wrapper}>
50             <TextInput
51                 placeholder='Nom'
52                 style={styles.textInput}
53                 onChangeText={(texte)=>this.setState({nom:texte })}
54             />
55             <TextInput
56                 placeholder='Prenom'
57                 style={styles.textInput}
58                 onChangeText={(texte)=>this.setState({prenom:texte })}
59             />
60             <Button
61                 title='Ajouter nom'
62                 onPress={() => { this.ajouter() }}
63             />
64             {/* Entete du tableau */}
65             <View style={styles.entete}>
66                 <View style={styles.cel}>
67                     <Text>Nom</Text>
68                 </View>
69                 <View style={styles.cel}>
70                     <Text>Prenom</Text>
71                 </View>
72             </View>
73             {/* Contenu du tableau */}

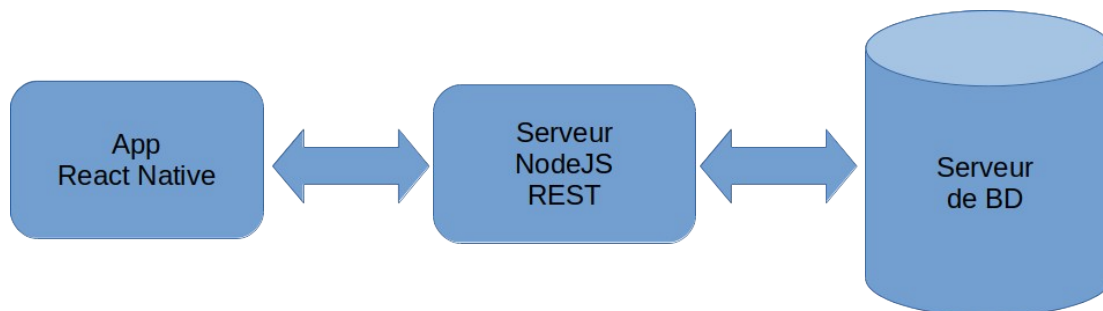
```

```

74         <FlatList
75             data={this.state.maBD}
76             keyExtractor={({item, index}) => {item._id}}
77             renderItem={({item})=>{return(this.renderRow(item))}}
78         />
79     </View>
80 )
81 }
82 }
83
84 const styles = StyleSheet.create({
85     wrapper: {
86         flex: 1,
87     },
88     textInput: {
89         borderWidth: 1,
90     },
91     entete: {
92         alignSelf: 'stretch',
93         flexDirection: 'row',
94         marginTop: 5,
95         backgroundColor: 'gray',
96     },
97     cel: {
98         flex: 1,
99         alignSelf: 'stretch',
100        paddingLeft: 5,
101        paddingTop: 2,
102        paddingBottom: 2,
103        borderWidth: 1,
104    },
105    ligne: {
106        alignSelf: 'stretch',
107        flexDirection: 'row',
108    }
109 });
110 });
111
112 export default Mongoose

```

Base de données distante



MQTT

Installer *react-native-paho-mqtt* dans le nouveau projet

```
npm install react-native-paho-mqtt --save
```

```
1 // components/mqtt.js
2
3 import React from 'react';
4 import { StyleSheet, View, Text, TextInput, Button } from 'react-native';
5 import { Client, Message } from 'react-native-paho-mqtt';
6
7 const myStorage = {
8   setItem: (key, item) => {
9     myStorage[key] = item;
10   },
11   getItem: (key) => myStorage[key],
12   removeItem: (key) => {
13     delete myStorage[key];
14   },
15 };
16
17 class mqtt extends React.Component {
18
19   static navigationOptions = {
20     headerTitle: "MQTT",
21   };
22
23   state = {
24     message: '',
25     reponse: '',
26   }
27
28   componentDidMount() {
29     this.client = new Client({ uri: 'ws://mqtt.eclipseprojects.io:80/
30                               mqtt', clientId: '', storage: myStorage });
31     this.client.connect()
32       .then(() => {
33         console.log('onConnect');
34         return this.client.subscribe('World');
35       })
36       .then(() => {
37         console.log('onSubscribe');
38       });
39
40     this.client.on('messageReceived', (message) => {
41       console.log(message.payloadString);
42       this.setState({ reponse: message.payloadString });
43     });
44   }
```

```

45
46   modifierMessage = (texte) => {
47     this.setState({ message: texte });
48   }
49
50   render() {
51     return (
52       <View style={styles.wrapper}>
53         <TextInput
54           placeholder='Message'
55           style={styles.textInput}
56           onChangeText={this.modifierMessage}
57         />
58         <Button
59           title='Envoyer'
60           onPress={() => {
61             //alert(this.state.message);
62             const message = new Message(this.state.message);
63             message.destinationName = 'World';
64             this.client.send(message);
65           }}
66         />
67         <Text>
68           Reponse: {this.state.reponse}
69         </Text>
70       </View>
71     )
72   }
73 }
74
75 const styles = StyleSheet.create({
76   wrapper: {
77     flex: 1,
78     alignItems: 'center',
79     justifyContent: 'center',
80   },
81   textInput: {
82     borderWidth: 1,
83     width: '50%'
84   }
85 });
86
87 export default mqtt

```

Publier sur Google Play

Référence :

- <https://reactnative.dev/docs/signed-apk-android>
- <https://medium.com/geekculture/react-native-generate-apk-debug-and-release-apk-4e9981a2ea51>

```
1 # Étape 1: À partir du répertoire racine du projet taper
2 # la commande suivante :
3
4 react-native bundle --dev false --platform android --entry-file index.js
5   --bundle-output ./android/app/build/intermediates/assets/debug/index.android.bundle
6   --assets-dest ./android/app/build/intermediates/res/merged/debug
7
8 # Étape 2: Aller dans le répertoire Android
9 cd android
10
11 # Étape 3: Créer le "debug build"
12 ./gradlew assembleDebug
13
14 # Étape 4: Créer le "release build"
15 ./gradlew assembleRelease
```

Le fichier "apk" qui peut être installé sur votre téléphone Android se trouve dans le répertoire `android/app/build/outputs/apk/release/`.

Références

- <https://reactnative.dev/docs/getting-started>
- <https://reactnavigation.org/docs/getting-started>
- <https://www.reactnative.express/>